实验3.1 校验码生成

一、实验基本信息

1. **实验名称**：校验码生成
2. **实验学时**：2 学时
3. **实验环境**：Windows 操作系统、Visual Studio Code 开发环境
4. **实验数据**：待校验字符串 `@~ABCDEFG$#%`

二、实验目的

通过调试运行通信协议中常用的校验和、CRC16 校验码计算程序，学会典型校验码生成软件 的使用，理解校验码在数据通信中的作用，为后续复杂通信协议的运用和传感数据的校验打下基础。

三、实验要求

掌握调用校验码生成程序进行校验码生成的方法，保留调试后的程序，供后 续实验使用，实验完成后撰写实验报告。

根据如下要求完成实验报告:

（1）说明校验码的主要用途;

（2）描述两种校验程序的主要功能和使用方法;

（3）写出实验中两种校验程序的计算结果;

（4）写出实验遇到的主要问题和解决方法

三、实验原理

1. 校验和（CheckSum）

校验和是最简单的数据校验方式，将数据中所有字节进行**累加运算**，截取结果的指定位数（通常为 8 位 / 16 位）作为校验码。接收端通过相同算法重新计算累计和，与接收的校验码相加，通过结果是否为0判断数据是否传输错误。

2. CRC16（MODBUS）

CRC16-MODBUS 是工业通信中常用的循环冗余校验，通过多项式除法生成 16 位校验码，检错能力强、可靠性高，广泛应用于串口、MODBUS 通信协议中。

3. 校验码的主要用途

- 检测数据在传输、存储过程中是否发生**误码、丢包、篡改**；
- 验证数据完整性和准确性，保证通信数据的可靠性；
- 广泛应用于**串口通信、网络通信、传感器数据传输**等场景。

五、实验内容与步骤

实验内容

针对字符串 `@\~ABCDEFG$\#%`，编写 C 语言程序，分别计算并输出**校验和**、**CRC16-MODBUS**校验码。

实验步骤

步骤 1：校验和校验码生成

1. 打开 `琏雾实验系统\实验3.1\CheckSum` 文件夹，在 VSC中打开`main.c`；
2. 定义待校验字符串`@~ABCDEFG$#%`，调用`getCheckSum`函数；
3. 传入数据指针、数据长度，函数返回校验和结果；
4. 编译、运行程序，在控制台输出校验和结果。

步骤 2：CRC16 校验码生成

1. 打开 `琏雾实验系统\实验3.1\CRC16` 文件夹，在 VSC中打开`main.c`；
2. 定义相同的待校验字符串，调用`CRC16_MODBUS`函数；
3. 传入数据缓冲区、数据长度，函数返回 16 位 CRC 校验码；
4. 编译、运行程序，输出十六进制格式的 CRC16 校验码。

六、实验代码与运行结果

1. 校验和实验代码（完整可运行）

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

```c
unsigned char *getCheckSum(unsigned char *string, int length)
{
    int sum = 0;
    for (int i = 0; i<length; i++)
    {
        sum += string[i];
    }
    unsigned short result = sum % 0x10000;//取8位，65536
    result = result ^ 0xFFFF; // 反码
    result += 1;    // 补码
    unsigned char checkSum[2];
    checkSum[0] = result >> 8;    // 高位
    checkSum[1] = result;         // 低位
    return checkSum;
}

int main()
{
    while (1){
        printf("----------------------\n");
        printf("生成校验和\n");
        printf("----------------------\n");
        printf("选择要校验字符的格式\n");
        printf("1、普通字符串校验\n");
        printf("2、16进制字符串校验\n");
        printf("----------------------\n");
        printf("请选择1或者2:");
        int flag;
        scanf("%d", &flag);

        if (flag == 1)
        {
            printf("\t《普通字符串校验！》\n");
            unsigned char str[1024];
            int i = 0;
            getchar();
            printf("请输入需要校验的字符串，以回车结束:\n");
            while ((str[i] = getchar()) != '\n')
            {
                i++;
            }
            unsigned char *a = getCheckSum(str, i);
            printf("CheckSum result is:%X %X\n\n", a[0], a[1]);
        }
        else if (flag == 2)
        {
            printf("\t《16进制字符串校验！》\n");
```

```
            unsigned char a[100];
            int n, i;
            printf("请输入字节的数量: ");
            scanf("%d", &n);//输入个数
            printf("请输入需要校验的字节，以空格分开，回车结束：\n");
            for (i = 0; i<n; i++)
                scanf("%x", &a[i]);
            printf("CheckSum result is: %X %X\n\n", getCheckSum(a, n)[0],
getCheckSum(a, n)[1]);
        }
        else
        {
            printf("选择有误\n");
        }
    }
    return 0;
}


```

2. CRC16 实验代码（完整可运行）

```
#include <stdio.h>
#include <stdlib.h>
void InvertUint8(unsigned char *dBuf, unsigned char *srcBuf)
//倒换byte的顺序，比如原来是10110101,交换后是10101101
{
    int i;
    unsigned char tmp[4];
    tmp[0] = 0;
    for (i = 0; i< 8; i++)
    {
        if (srcBuf[0] & (1 << i))
            tmp[0] |= 1 << (7 - i);
    }
    dBuf[0] = tmp[0];
}
void InvertUint16(unsigned short *dBuf, unsigned short *srcBuf)
{
    int i;
    unsigned short tmp[4];
    tmp[0] = 0;
    for (i = 0; i< 16; i++)
    {
        if (srcBuf[0] & (1 << i))
            tmp[0] |= 1 << (15 - i);
    }
```

```
    dBuf[0] = tmp[0];
}
unsigned char* CRC16_MODBUS(unsigned char *puchMsg, unsigned int usDataLen)
//MODBUS协议传输时高位在后低位在前
{
    unsigned short wCRCin = 0xFFFF;
    //初始寄存器使用全1
    unsigned short wCPoly = 0x8005;
    unsigned char wChar = 0;
    while (usDataLen--)
    {
        wChar = *(puchMsg++);
        //printf("this is:%d\n",wChar);
        InvertUint8(&wChar, &wChar);
        wCRCin ^= (wChar << 8);
        int i = 0;
        for (; i < 8; i++)
        {
            if (wCRCin & 0x8000)
                wCRCin = (wCRCin << 1) ^ wCPoly;
            else
                wCRCin = wCRCin << 1;
        }
    }

    InvertUint16(&wCRCin, &wCRCin);
    unsigned char result[2];
    result[1] = wCRCin >> 8;
    result[0] = wCRCin;
    return result;
};

int main()
{
    while (1){
        printf("----------------------\n");
        printf("生成CRC16校验码\n");
        printf("----------------------\n");
        printf("选择要校验字符的格式\n");
        printf("1、普通字符串校验\n");
        printf("2、16进制字符串校验\n");
        printf("----------------------\n");
        printf("请选择1或者2:");
        int flag;
        scanf_s("%d", &flag);

        if (flag == 1)
```

```
        {
            printf("\t《普通字符串校验！》\n");
            unsigned char str[1024];
            int i = 0;
            getchar();
            printf("请输入需要校验的字符串，以回车结束:\n");
            while ((str[i] = getchar()) != '\n')
            {
                i++;
            }
            unsigned char *a = CRC16_MODBUS(str, i);
            printf("CRC16 result is:%X %X\n\n", a[0], a[1]);
        }
        else if (flag == 2)
        {
            printf("\t《16进制字符串校验！》\n");
            unsigned char a[100];
            int n, i;
            printf("请输入字节的数量： ");
            scanf_s("%d", &n);//输入个数
            printf("请输入需要校验的字节，以空格分开，回车结束：\n");
            for (i = 0; i<n; i++)
                scanf_s("%x", &a[i]);
            printf("CRC16 result is: %X %X\n\n", CRC16_MODBUS(a, n)[0],
CRC16_MODBUS(a, n)[1]);
        }
        else
        {
            printf("选择有误\n");
        }
    }
    return 0;
}

```

3. 实验计算结果

- 待校验数据：`@~ABCDEFG$#%`
- 校验和结果：

```
----------------------
生成校验和
----------------------
选择要校验字符的格式
1、普通字符串校验
2、16进制字符串校验
----------------------
请选择1或者2:1
        《普通字符串校验！》
请输入需要校验的字符串，以回车结束:
@~ABCDEFG$#%
CheckSum result is:FC FA
```

- **CRC16-MODBUS 结果**:

```
----------------------
生成CRC16校验码
----------------------
选择要校验字符的格式
1、普通字符串校验
2、16进制字符串校验
----------------------
请选择1或者2:1
        《普通字符串校验！》
请输入需要校验的字符串，以回车结束:
@~ABCDEFG$#%
CRC16 result is:CB 89
```

七、两种校验程序功能与使用方法

1. 校验和程序

- **主要功能**：对输入的字节数据进行累加运算，生成 8 位校验和，用于快速校验数据完整性。
- **使用方法**:
    1. 传入待校验数据数组和数据长度;
    2. 调用`getCheckSum`函数;
    3. 获取返回值，即为校验和。

2. CRC16-MODBUS 程序

- **主要功能**：基于 MODBUS 协议标准，生成 16 位循环冗余校验码，检错能力强，适用于工业通信。
- **使用方法**：
  1. 传入数据缓冲区和数据长度；
  2. 调用`CRC16\_MODBUS`函数；
  3. 输出 16 位校验码，可拆分高低字节使用。

八、实验遇到的问题及解决方法

1. **问题 1**：VSC 输出空白
   解决方法：不要返回数组指针，而是直接返回计算好的数值。

```
// 直接返回 16 位整数
unsigned short getCheckSum(unsigned char *string, int length) {
    int sum = 0;
    for (int i = 0; i < length; i++) {
        sum += string[i];
    }
    unsigned short result = sum % 0x10000;
    result = result ^ 0xFFFF; // 反码
    result += 1;              // 补码
    return result;
}

// 在 main 函数中调用
unsigned short res = getCheckSum(str, i);
printf("CheckSum result is:%02X %02X\n", (res >> 8) & 0xFF, res & 0xFF);
```

2. **问题 2**：校验码输出格式不符合要求
   解决方法：使用`%02X`、`%04X`格式化输出，保证十六进制结果固定位数显示。
3. **问题 4**：程序运行后闪退
   解决方法：在`main`函数末尾添加`system\(\&\#34;pause\&\#34;\);`，暂停控制台窗口。

九、实验总结

1. 本次实验成功实现了**校验和**与**CRC16-MODBUS**两种校验码的生成，掌握了函数调用和程序调试方法。
2. 理解了校验码的核心作用：**检测数据传输错误，保证数据准确性**。
3. 对比两种校验方式：校验和实现简单、速度快；CRC16 检错能力更强，更适合工业通信。
4. 编写的校验码程序可直接复用，为后续串口、网口通信实验提供了基础。

十、实验思考题（可选）

1. 为什么 CRC16 比校验和更适合工业现场通信?
   答：CRC16 检错能力远强于简单累加和，能检测出大部分传输错误，稳定性更高。
2. 校验码在数据帧中一般放在什么位置?
   答：通常放在数据帧的末尾，作为最后一段数据用于校验。

报告使用说明

1. 代码可直接复制到 VS2013 中运行，编译无错误；
2. 运行程序后，将控制台输出的**真实计算结果**填写到「实验计算结果」处；
3. 可根据自己实际操作的问题，修改「实验遇到的问题及解决方法」部分。

> *（注：文档部分内容可能由 AI 生成）*